Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

TALLER 5: DOCKER NIVEL INTERMEDIO

Propósito:

Este taller tiene como propósito profundizar en el uso de Docker mediante el estudio de mecanismos de **persistencia de datos, redes entre contenedores y despliegue de aplicaciones compuestas**.

A diferencia del taller anterior, donde se exploraron los comandos básicos para ejecutar contenedores, en este laboratorio se abordarán conceptos intermedios del ecosistema Docker que son fundamentales en entornos reales de desarrollo y despliegue de aplicaciones.

En particular, se estudiará cómo los contenedores manejan el almacenamiento persistente mediante **volúmenes**, cómo múltiples contenedores pueden interactuar entre sí a través de **redes virtuales**, y cómo aplicaciones reales pueden ser desplegadas mediante la integración de servicios independientes.

Contexto:

Una de las características fundamentales de los contenedores es que son **efímeros por naturaleza**. Cuando un contenedor se elimina, el sistema de archivos asociado al contenedor también desaparece. Esto significa que cualquier dato generado durante la ejecución de la aplicación puede perderse si no se utiliza un mecanismo externo de almacenamiento.

En aplicaciones reales, como bases de datos o sistemas de gestión de contenido, esta situación representa un problema crítico, ya que los datos deben mantenerse disponibles incluso cuando los contenedores son detenidos, reiniciados o reemplazados.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Docker resuelve este problema mediante el uso de **volúmenes**, que permiten desacoplar los datos persistentes del ciclo de vida de los contenedores.

En este taller se analizará un caso práctico utilizando **PostgreSQL ejecutándose en un contenedor Docker**. Inicialmente se observará cómo los datos se pierden al eliminar el contenedor. Posteriormente se introducirá el uso de volúmenes para mantener los datos persistentes.

Finalmente, se integrará el servidor de base de datos PostgreSQL con un contenedor de **Drupal**, simulando una arquitectura básica de aplicación web compuesta por múltiples servicios.

Objetivos de aprendizaje:

Al finalizar este taller el estudiante debería ser capaz de:

1. Comprender el concepto de persistencia de datos en contenedores.
2. Explicar el funcionamiento de los volúmenes en Docker y su relación con el almacenamiento del sistema anfitrión.
3. Ejecutar contenedores que utilicen almacenamiento persistente mediante volúmenes.
4. Analizar la diferencia entre datos almacenados dentro del contenedor y datos almacenados en volúmenes externos.
5. Configurar redes de contenedores para permitir la comunicación entre servicios.
6. Integrar múltiples contenedores para desplegar una aplicación compuesta (Drupal + PostgreSQL).
7. Comprender cómo estos mecanismos son utilizados en entornos reales de despliegue de aplicaciones.

Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Parte 1- Contendores efímeros

Los contenedores en Docker son diseñados para ser **desechables**. Esto significa que cuando un contenedor es eliminado, todos los datos almacenados dentro de su sistema de archivos también desaparecen.

En el caso de aplicaciones como bases de datos, este comportamiento puede representar un problema importante si no se utilizan mecanismos de almacenamiento persistente.

Para observar este comportamiento se ejecutará un servidor **PostgreSQL dentro de un contenedor Docker** (puede ser descargado del repositorio git del curso creado por el profe).

Paso 1: Ejecución básica de PostgreSQL

La forma más simple de ejecutar un servidor PostgreSQL en Docker es mediante el siguiente comando:

```
docker container run --name lab-postgres-server -e POSTGRES_PASSWORD=passwd -d
postgres:9.6.1
```

Estos comandos realizan las siguientes acciones:

- -d postgres:9.6.1: Descarga la imagen de PostgreSQL si no se encuentra disponible localmente.
- --name: Crea un contenedor llamado **lab-postgres-server**.
- POSTGRES_PASSWORD=passwd: Configura la contraseña del usuario administrador. Recuerden remplazarla por un valor que se acuerden y eviten usar contraseñas por defecto como postgres, por temas de seguridad.
- docker container run: Ejecuta el contenedor en segundo plano.

Paso 2: Verificación e inspección del contenedor

Primero vamos a listar los contenedores en ejecución. No olvide evidenciar con un pantallazo este paso.

```
docker container ls
```

Recuerden que Docker permite inspeccionar la configuración interna de un contenedor mediante el comando:

```
docker inspect postgres-server
```

Por lo tanto, analice la información mostrada por el comando (**Evidencia requerida)**

Preguntas

1. ¿Cuál es el **ID del contenedor**?
2. ¿Qué **imagen** se está utilizando?
3. ¿Qué **variables de entorno** fueron configuradas?

Paso 3: Conexión al contenedor

Para interactuar con el servidor de base de datos es necesario ingresar al contenedor. Primero, ejecute el siguiente comando. Evidencielo con un pantallazo.

```
docker container exec -it --user postgres lab-postgres-server bash
```

Una vez dentro del contenedor, ejecute el cliente de PostgreSQL.

```
psql
```

Para listar las bases de datos disponibles usar el siguiente comando. No olvidar evidenciar estos dos últimos pasos con un solo pantallazo.

```
\l
```

Evidencia requerida

Tomar un **pantallazo** donde se observe el contenedor lab-postgres-server en ejecución.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Paso 4: Creación de una base de datos

Dentro del cliente de PostgreSQL ejecute:

```
create database demo;
```

Luego liste nuevamente las bases de datos:

```
\l
```

Observe que ahora aparece una base de datos llamada **demo**. Para salir del cliente de PostgreSQL:

```
\q
```

Luego salir del contenedor:

```
exit
```

Evidencia requerida

Tomar un pantallazo del resultado del comando y que se observe la base de datos demo.

Paso 5: Finalización del contenedor

Ahora detenga y elimine el contenedor:

```
docker container stop postgres-server
docker container rm postgres-server
```

Evidencia requerida

Tomar un **pantallazo** del resultado del comando.

Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Ejercicio 1

Ejecute nuevamente el contenedor PostgreSQL utilizando el mismo comando inicial. Luego, ingrese nuevamente al contenedor y liste las bases de datos disponibles.

Preguntas de análisis:

- ¿La base de datos demo sigue estando presente?
- Explique por qué la base de datos se mantiene o desaparece.
- ¿Qué relación existe entre este comportamiento y el sistema de archivos del contenedor?
- ¿Dónde se almacenan físicamente los datos generados por el contenedor en el sistema anfitrión?

Parte 2- Persistencia de los datos mediante volúmenes

En la sección anterior se observó que los contenedores Docker son **efímeros**, lo que significa que los datos generados dentro de un contenedor desaparecen cuando este es eliminado. Para resolver este problema, Docker proporciona un mecanismo llamado **volúmenes**.

Un volumen es un espacio de almacenamiento administrado por Docker que permite **persistir datos fuera del sistema de archivos del contenedor**. De esta forma, incluso si el contenedor es eliminado, los datos permanecen almacenados en el sistema anfitrión (es decir, que separa un espacio del disco duro para utilizarlo como medio de almacenamiento de la información).

En esta sección se utilizará un volumen Docker para almacenar los datos de una base de datos PostgreSQL.

Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Paso 1: Creación de un volumen

Primero cree un volumen en Docker utilizando el siguiente comando:

```
docker volume create postgres-vol
```

Paso 2: Verificación del volumen

Liste los volúmenes existentes utilizando el comando:

```
docker volume ls
```

Evidencia requerida

Tome un **pantallazo** donde se observe el volumen postgres-vol.

Paso 3: Inspección del volumen

Para observar la configuración del volumen ejecute:

```
docker volume inspect postgres-vol
```

Este comando mostrará información sobre el volumen, incluyendo la **ruta física donde se almacenan los datos en el sistema anfitrión**.

Evidencia requerida

Tome un **pantallazo** del resultado del comando y detalle lo que observa en consola.

Preguntas de análisis

1. ¿Cuál es la ruta del volumen en el sistema anfitrión?
2. ¿Qué driver está utilizando el volumen?

Paso 4: Observación del sistema anfitrión

Ahora observe directamente el sistema de archivos del host. Ejecute:

```
sudo ls /var/lib/docker/volumes
```

Esto permite identificar dónde Docker almacena físicamente los datos asociados a los volúmenes.

Evidencia requerida

Tome un **pantallazo** donde se observe el volumen postgres-vol.

Paso 5: Ejecución de PostgreSQL usando un volumen

Ahora se ejecutará nuevamente PostgreSQL, pero utilizando el volumen creado anteriormente.

Ejecute:

```
docker container run \
--name lab-postgres-server \
-e POSTGRES_PASSWORD=passwd \
-e PGDATA=/var/lib/postgresql/data \
-v postgres-vol:/var/lib/postgresql/data \
-d postgres:9.6.2
```

En este comando se observan dos elementos importantes:

- PGDATA que indica la ubicación donde PostgreSQL almacena sus datos.
- -v postgres-vol:/var/lib/postgresql/data que conecta esa ruta con el volumen creado (postgres-vol).

De esta forma, los datos de la base de datos serán almacenados **fuera del contenedor**.

Evidencie con un pantallazo lo realizado.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Paso 6: Interacción con la base de datos

Ingrese al contenedor como lo hizo en el Paso 3 de la primera parte del taller (Conexión al contenedor)

docker container exec -it --user postgres lab-postgres-server bash

Evidencia requerida

Tome un **pantallazo** del listado de bases de datos.

Paso 7: Creación de una base de datos

Ejecute:

```
create database demo;
```

Verifique nuevamente:

```
\l
```

Evidencia requerida

Tome un **pantallazo** donde se observe que la base de datos demo está creada nuevamente.

Paso 8: Eliminación del contenedor

Salga del contenedor y ejecute:

```
docker container stop postgres-server
docker container rm postgres-server
```

Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Ejercicio 2

1. Ejecute nuevamente el contenedor PostgreSQL utilizando el mismo comando de esta sección.
2. Ingrese al contenedor con el comando

```
docker container exec -it --user postgres postgres-server bash
```

3. Ejecute el cliente PostgreSQL:

```
psql
```

4. Liste las bases de datos:

```
\l
```

Preguntas de análisis

Responda las siguientes preguntas:

1. ¿La base de datos demo sigue existiendo después de eliminar el contenedor?
2. Explique por qué los datos **ya no desaparecen** al eliminar el contenedor.
3. ¿Cuál es el papel del volumen postgres-vol en este comportamiento?
4. ¿Qué relación existe entre los volúmenes Docker y el sistema de archivos del sistema anfitrión?

Paso 9: Limpieza del entorno

Finalmente elimine los recursos utilizados:

```
docker container stop postgres-server
docker container rm postgres-server
docker volume rm postgres-vol
```

Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Parte 3- Redes de contenedores

En los ejercicios anteriores se observó cómo ejecutar contenedores y cómo almacenar datos persistentes utilizando volúmenes.

Sin embargo, las aplicaciones reales rara vez se ejecutan como un único contenedor. Normalmente están compuestas por **múltiples servicios que deben comunicarse entre sí**, por ejemplo:

- servidor web
- servidor de base de datos
- servicios de aplicación

Docker permite que estos servicios se comuniquen mediante **redes virtuales de contenedores**.

En esta sección se creará una red Docker para permitir la comunicación entre:

- un contenedor **PostgreSQL**
- un contenedor **Drupal**

Paso 1: Creación de una red Docker

Primero cree una red llamada **drupal-postgres**:

```
docker network create drupal-postgres
```

Evidencie el resultado con un pantallazo.

Paso 2: Verificación de redes

Liste las redes existentes:

```
docker network ls
```

Evidencia requerida

Tome un **pantallazo** donde se observe la red drupal-postgres y explique la información que ve en pantalla

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Paso 3: Inspección de la red

Ejecute:

```
docker network inspect drupal-postgres
```

Este comando permite observar:

- configuración de la red
- subred asignada
- contenedores conectados

Evidencia requerida

Tome un **pantallazo** del resultado.

Paso 4: Ejecución del contenedor PostgreSQL

Ejecute el servidor PostgreSQL conectado a la red creada.

```
docker container run \
--name lab-postgres-server \
--network drupal-postgres \
-e POSTGRES_PASSWORD=passwd \
-d postgres:10
```

Paso 5: Verificación del contenedor

Liste los contenedores en ejecución:

```
docker container ls
```

Evidencia requerida

Tome un **pantallazo** donde se observe el contenedor lab-postgres-server.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Paso 6: Ejecución del contenedor Drupal

Ahora ejecute un contenedor Drupal.

```
docker container run \
--name drupal \
--network drupal-postgres \
-p 8080:80 \
-d drupal:9
```

Este comando:

- crea un contenedor llamado **drupal**
- lo conecta a la red drupal-postgres
- expone el puerto **80 del contenedor en el puerto 8080 del host**

Paso 7: Verificación de contenedores

Ejecute:

```
docker container ls
```

Evidencia requerida

Tome un **pantallazo** donde se observen los contenedores:

- postgres-server
- drupal

Observación de logs

En caso de errores, revise los logs de los contenedores.

Logs del contenedor PostgreSQL:

```
docker logs lab-postgres-server
```

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Logs del contenedor Drupal:

```
docker logs drupal
```

Evidencia requerida

Tome un **pantallazo** de los logs.

Paso 8: Acceso a Drupal

Abra su navegador y acceda a la ruta http://localhost:8080. Se iniciará el proceso de instalación de Drupal. Cuando se accede por primera vez, se despliegan las instrucciones de configuración

Configuración de la base de datos

Durante la instalación seleccione:

Tipo de base de datos

PostgreSQL

Parámetros de conexión

Database name: postgres
Username: postgres
Password: passwd

En la sección de **configuración avanzada**:

Host: lab-postgres-server

Observe que se utiliza el **nombre del contenedor como hostname**, gracias a la red Docker creada.

Evidencia requerida

Tome un **pantallazo** donde se observe la interfaz de instalación de Drupal conectándose al servidor PostgreSQL.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Ejercicio 3

Preguntas de análisis

Responda las siguientes preguntas:

1. ¿Por qué Drupal puede conectarse a PostgreSQL usando el nombre postgres-server?
2. ¿Qué papel cumple la red Docker drupal-postgres?
3. ¿Qué ocurriría si ambos contenedores no estuvieran conectados a la misma red?
4. ¿Qué ventajas ofrece este modelo de despliegue frente a ejecutar ambos servicios directamente en el sistema operativo anfitrión?

Paso 9: Limpieza del entorno

Finalmente elimine los contenedores y la red creada.

```
docker container rm -f postgres-server drupal
docker network rm drupal-postgres
```

Parte 4- Docker Compose

En las secciones anteriores se ejecutaron contenedores manualmente utilizando múltiples comandos docker run. Aunque este enfoque permite comprender el funcionamiento de Docker, en escenarios reales las aplicaciones suelen estar compuestas por múltiples servicios que deben ejecutarse de forma coordinada.

Docker Compose permite definir y ejecutar aplicaciones multi-contenedor mediante un archivo declarativo llamado: **docker-compose.yml**

Este archivo describe:

- los servicios de la aplicación
- las redes
- los volúmenes
- las dependencias entre contenedores

De esta forma, los diferentes elementos que pueden componer a una aplicación como son el backend, el frontend y la base de datos, pueden desplegarse utilizando un solo comando.

Paso 1: Creación del entorno de trabajo

Cree un directorio para el stack de Drupal:

```
mkdir drupal-stack
cd drupal-stack
```

Paso 2: Creación del archivo docker-compose.yml

Cree un archivo llamado: docker-compose.yml con el siguiente contenido:

```
version: '3.8'

services:

  postgres:
    image: postgres:10
    container_name: lab-postgres-server
    environment:
      POSTGRES_PASSWORD: passwd
    volumes:
      - postgres-vol:/var/lib/postgresql/data
    networks:
      - drupal-network

  drupal:
    image: drupal:9
    container_name: drupal
    ports:
      - "8080:80"
    depends_on:
```

```
      - postgres
    networks:
      - drupal-network

volumes:
  postgres-vol:

networks:
  drupal-network:
```

Este archivo define:

- un servicio **postgres**
- un servicio **drupal**
- un volumen para persistencia de datos
- una red compartida entre ambos contenedores

Paso 3: Despliegue de la aplicación

Ejecute el siguiente comando:

```
docker compose up -d
```

Docker Compose realizará automáticamente las siguientes acciones:

1. Creará la red drupal-network
2. Creará el volumen postgres-vol
3. Lanzará el contenedor postgres
4. Lanzará el contenedor drupal

Verificación de contenedores

Liste los contenedores en ejecución:

```
docker container ls
```

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Evidencia requerida

Tome un pantallazo donde se observen los contenedores:

- postgres-server
- drupal

Paso 4: Verificación de redes y volúmenes

Liste las redes creadas:

```
docker network ls
```

Liste los volúmenes:

```
docker volume ls
```

Evidencia requerida

Tome un pantallazo donde se observe:

- la red drupal-network
- el volumen postgres-vol

Paso 5: Acceso a Drupal

Abra el navegador en http://localhost:8080. Durante la instalación de Drupal utilice los siguientes parámetros:

Database type: PostgreSQL
Database name: postgres
Username: postgres
Password: passwd
Host: postgres

Observe que el contenedor Drupal puede conectarse al contenedor PostgreSQL utilizando el **nombre del servicio** definido en Docker Compose.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Paso 6: Inspección del entorno creado por Compose

Docker Compose crea automáticamente recursos asociados al proyecto. Puede inspeccionar el entorno con:

```
docker compose ps
```

También puede ver los logs de los servicios:

```
docker compose logs
```

Paso 7: Detener el entorno

Para detener la aplicación ejecute:

```
docker compose down
```

Este comando detiene y elimina los contenedores definidos en el archivo Compose.

Ejercicio 4

Preguntas de análisis

1. ¿Qué ventajas ofrece Docker Compose frente a ejecutar múltiples comandos docker run?
2. ¿Por qué el contenedor Drupal puede conectarse a PostgreSQL utilizando el nombre postgres?
3. ¿Qué recursos (redes y volúmenes) crea automáticamente Docker Compose?

Parte 5- Orquestación local con Docker Compose

Docker Swarm es una herramienta de orquestación que permite gestionar contenedores como **servicios distribuidos**.

Mientras que Docker Compose ejecuta contenedores en un solo host, Docker Swarm permite desplegar aplicaciones en múltiples nodos formando un **cluster de contenedores (agrupación de contenedores)**. En Swarm, las aplicaciones se

despliegan como **stacks**, que se pueden entender como una colección de servicios que se despliegan a partir de un archivo similar a docker-compose.yml.

Paso 1: Inicializar Docker Swarm

Inicialice el modo Swarm:

```
docker swarm init
```

Este comando convierte el host actual en un **nodo administrador del cluster**.

Paso 2: Verificación del cluster

Liste los nodos del cluster:

```
docker node ls
```

Evidencia requerida

Tome un pantallazo donde se observe el nodo administrador.

Paso 3: Despliegue del stack

Utilice el mismo archivo docker-compose.yml para desplegar el stack.

Ejecute:

```
docker stack deploy -c docker-compose.yml drupal-stack
```

Esto creará un stack llamado:

drupal-stack

Paso 4: Verificación del stack

Liste los servicios del stack:

```
docker stack services drupal-stack
```

También puede listar los contenedores en ejecución:

```
docker service ls
```

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Paso 5: Escalamiento de servicios

Una de las principales ventajas de Swarm es la capacidad de escalar servicios.

Ejecute el siguiente comando para escalar el servicio Drupal:

```
docker service scale drupal-stack_drupal=3
```

Esto creará **tres contenedores ejecutando Drupal simultáneamente**.

Paso 6: Verificación del escalamiento

Liste los contenedores:

```
docker container ls
```

Evidencia requerida

Tome un pantallazo donde se observen múltiples contenedores Drupal ejecutándose.

Ejercicio 5

Preguntas de análisis

1. ¿Qué diferencia existe entre ejecutar contenedores con docker run y ejecutar servicios con docker service?
2. ¿Qué ventaja ofrece la capacidad de escalar servicios?
3. ¿Cómo se relaciona este modelo con conceptos de **computación distribuida**?

Paso 7: Limpieza del entorno

Para eliminar el stack ejecute:

```
docker stack rm drupal-stack
```

Luego desactive el modo swarm:

```
docker swarm leave --force
```